

Doi indici peste două tablouri sortate

Subiectul III · șapte cerințe pe același mecanism · Victor

Cadrul comun, valabil la toate cele șapte: `a` este un tablou unidimensional cu `n` numere naturale de cel mult 4 cifre fiecare, `b` un tablou cu `m` numere naturale de cel mult 4 cifre fiecare, $1 < n < 100$, $1 < m < 100$. **Amândouă sunt deja ordonate crescător. Nu le sorta.**

Constrângerea care dă tot rostul fișei: o singură parcurgere — fiecare element al lui `a` și al lui `b` se citește cel mult o dată. **Fără bucle imbricate**, fără a căuta o valoare într-un tablou cu o a doua buclă.

Ce ai voie: `int`, `long`, `bool` · tablouri unidimensionale · funcții cu parametri `int a[]` și `int &x` · `for`, `while`, `do...while`, `if`, `else`, `break` · `cin`, `cout`.

Încă NU folosim: `struct`, `string`, `vector`, recursivitate, `<algorithm>`, tablouri bidimensionale.

Antetul îl scrii tu. Enunțul îți dă numele subprogramului și al parametrilor, ca la bac — tipul returnat, ordinea și ce e referință hotărâști tu.

Partea 1 — mecanismul de bază

Subprogramul `unire`

1

Subprogramul `unire` are șase parametri, `a`, `n`, `b`, `m`, `c` și `p`. Subprogramul construiește în tabloul `c` toate cele `n + m` elemente ale tablourilor `a` și `b`, ordonate crescător, și furnizează prin parametrul `p` numărul de elemente din `c`. Valorile care apar în ambele tablouri apar de câte ori apar în total.

```
a = (2, 5, 9), b = (1, 5, 7, 12)
-> c = (1, 2, 5, 5, 7, 9, 12), p = 7
```

```
a = (1, 2, 3), b = (7, 8)
-> c = (1, 2, 3, 7, 8), p = 5
```

Subprogramul `comune`

2

Subprogramul `comune` are patru parametri, `a`, `n`, `b` și `m`, și afișează pe ecran, în ordine crescătoare, separate prin câte un spațiu, valorile care apar în amândouă tablourile. Fiecare astfel de valoare se afișează o singură dată. Dacă nu există nicio valoare comună, subprogramul afișează pe ecran mesajul `NU EXISTA`.

```
a = (2, 5, 5, 9, 14), b = (1, 5, 9, 20)    -> 5 9
a = (2, 4),          b = (1, 3)          -> NU EXISTA
a = (3, 3, 3),       b = (3, 3)          -> 3
```

Subprogramul `doarInA`

3

Subprogramul `doarInA` are patru parametri, `a`, `n`, `b` și `m`, și afișează pe ecran, în ordine crescătoare, separate prin câte un spațiu, valorile care apar în tabloul `a` și nu apar în tabloul `b`. Fiecare astfel de valoare se afișează o singură dată. Dacă nu există nicio astfel de valoare, subprogramul afișează pe ecran mesajul `NU EXISTA`.

```
a = (2, 5, 5, 9, 14), b = (1, 5, 9, 20)    -> 2 14
a = (4, 6),          b = (4, 6, 8)        -> NU EXISTA
a = (1, 2, 3),       b = (9)              -> 1 2 3
```

Partea 2 — aceeași parcurgere, o decizie în plus

Subprogramul **reuniune**

4

Subprogramul **reuniune** are șase parametri, **a**, **n**, **b**, **m**, **c** și **p**. Subprogramul construiește în tabloul **c**, în ordine crescătoare, valorile care apar în cel puțin unul dintre tablourile **a** și **b**, fiecare valoare o singură dată, și furnizează prin parametrul **p** numărul de elemente din **c**.

```
a = (2, 5, 5, 9), b = (1, 5, 7, 9)
-> c = (1, 2, 5, 7, 9), p = 5
```

```
a = (4, 4, 4), b = (4, 4)
-> c = (4), p = 1
```

Subprogramul **alternant**

5

Subprogramul **alternant** are patru parametri, **a**, **n**, **b** și **m**. Tabloul **a** conține **n** numere **pare**, tabloul **b** conține **m** numere **impare**, amândouă ordonate crescător. Subprogramul afișează pe ecran, în ordine crescătoare, separate prin câte un spațiu, un șir format dintr-un număr **maxim** de elemente care aparțin cel puțin unuia dintre tablouri, astfel încât orice două elemente aflate pe poziții consecutive să fie de paritate diferită.

```
a = (2, 4, 8, 10, 14), b = (3, 5, 11) -> 2 3 4 5 8 11 14
a = (2, 4, 6), b = (1) -> 1 2
a = (10, 20), b = (1, 3, 5) -> 1 10
a = (2, 4), b = (1, 3) -> 1 2 3 4
```

Partea 3 — provocări

Subprogramul **perechi**

6

Subprogramul **perechi** are cinci parametri, **a**, **n**, **b**, **m** și **s**. Subprogramul afișează pe ecran toate perechile formate dintr-un element **x** al tabloului **a** și un element **y** al tabloului **b**, cu proprietatea că $x + y = s$. Perechile se afișează câte una pe linie, sub forma **x y**, în ordinea crescătoare a valorilor lui **x**. Dacă nu există nicio astfel de pereche, subprogramul afișează pe ecran mesajul **NU EXISTA**.

Constrângerea de o singură parcurgere rămâne valabilă și aici.

```
a = (1, 3, 6, 8, 11), b = (2, 4, 5, 9), s = 10
-> 1 9
    6 4
    8 2
```

```
a = (1, 2), b = (3, 4), s = 100 -> NU EXISTA
```

Subprogramul `alKlea`

Subprogramul `alKlea` are cinci parametri, `a`, `n`, `b`, `m` și `k`, unde $1 \leq k \leq n + m$. Subprogramul returnează al `k`-lea element, în ordine crescătoare, dintre toate cele `n + m` elemente ale tablourilor `a` și `b` luate împreună.

Nu ai voie să construiești tabloul interclasat — niciun al treilea tablou.

```
a = (2, 5, 9), b = (1, 5, 7, 12), k = 4    -> 5
a = (2, 5, 9), b = (1, 5, 7, 12), k = 1    -> 1
a = (2, 5, 9), b = (1, 5, 7, 12), k = 7    -> 12
a = (1, 2, 3), b = (4, 5), k = 5          -> 5
```

Înainte să zici că e gata — trece fiecare subprogram prin lista asta:

- L-ai rulat pe **toate** exemplele din enunț, nu doar pe primul?
- Merge cu `m = 1`? Dar cu `n = 1`?
- Merge când toate elementele lui `a` sunt mai mici decât toate ale lui `b`? Dar invers?
- Merge când `a` și `b` au valori egale — și când au aceeași valoare de mai multe ori?
- Se termină **tot** ce trebuie scris pe ecran, sau se oprește cu un element înainte de capăt?
- Numără de câte ori citești fiecare element: dacă vreunul e citit de două ori, n-ai o singură parcurgere.
- Verifică ce înseamnă `i` și `j` în **fiecare** loc unde le folosești: arată *pe* elementul consumat, sau *după* el? Alege o variantă și ține-o peste tot în subprogram.

Un fișier per exercițiu. Rulează, nu doar compila.